

WhatsUT

Documentação Técnica do Sistema

Sistema de Comunicação Interpessoal
baseado em Java RMI

Universidade Tecnológica Federal do Paraná
Sistemas Distribuídos

Sumário

1. Visão geral	3
2. Arquitetura em três camadas	3
3. O que é RMI (explicação simplificada)	5
4. As classes principais	6
5. Fluxo de uma mensagem em grupo	8
6. O padrão Callback (RMI bidirecional)	9
7. Threads — quem faz o quê	10
8. Persistência e estado	12
9. Heartbeat e detecção de clientes mortos	12
10. Como executar o sistema	13

1. Visão geral

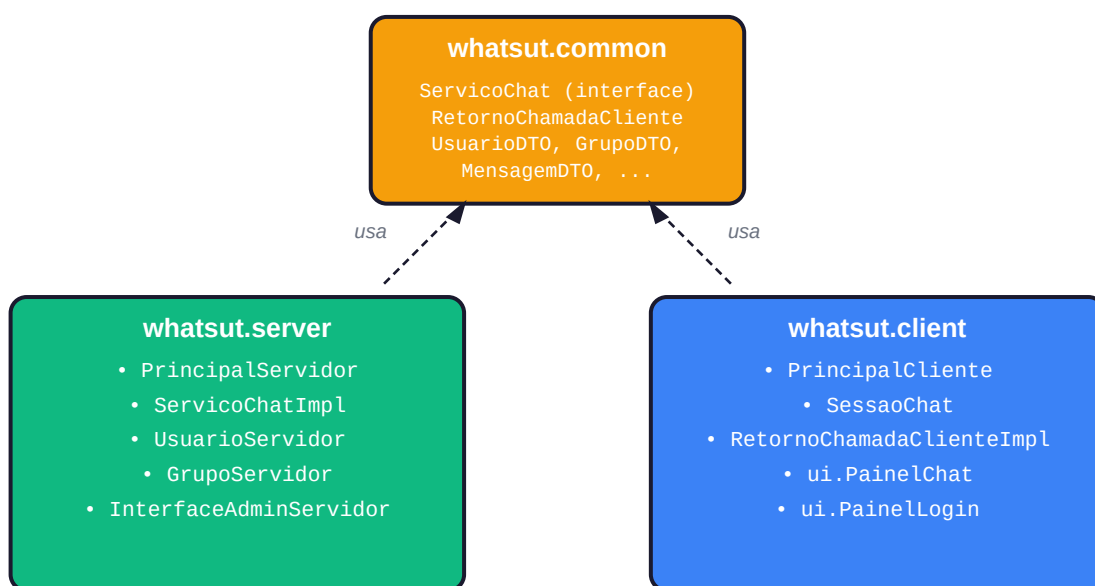
O **WhatsUT** é um sistema de chat estilo WhatsApp construído inteiramente em Java. Ele permite que vários usuários se cadastrem, façam login, troquem mensagens privadas, criem grupos com aprovação de entrada, enviem arquivos e sejam moderados por um administrador.

O que torna o sistema interessante do ponto de vista de Sistemas Distribuídos é que ele **não usa HTTP, REST nem WebSockets**. A comunicação entre cliente e servidor é feita via **Java RMI (Remote Method Invocation)**, uma tecnologia que permite chamar métodos Java em outro processo — mesmo em outra máquina — como se fossem locais.

Não há banco de dados: tudo fica em memória no servidor (estruturas thread-safe). A interface gráfica usa **Java Swing**, mas neste documento vamos focar no que acontece no **backend** e na **rede**.

2. Arquitetura em três camadas

O projeto é dividido em três pacotes Java, organizados por responsabilidade:



common é compartilhado entre cliente e servidor — é o 'contrato' RMI

Figura 1 — Pacotes do projeto e suas relações

whatsut.common é o coração do contrato RMI. Contém as **interfaces remotas** (`ServicoChat` e `RetornoChamadaCliente`) e as classes **DTO** (Data Transfer Object) — objetos simples que viajam pela rede serializados como bytes.

whatsut.server contém a implementação do serviço e suas classes auxiliares (modelos internos de usuário/grupo, painel admin). Só roda no servidor.

whatsut.client contém a aplicação do usuário final, incluindo o callback que *também é um objeto remoto* — o servidor liga de volta para ele.

Visão de execução

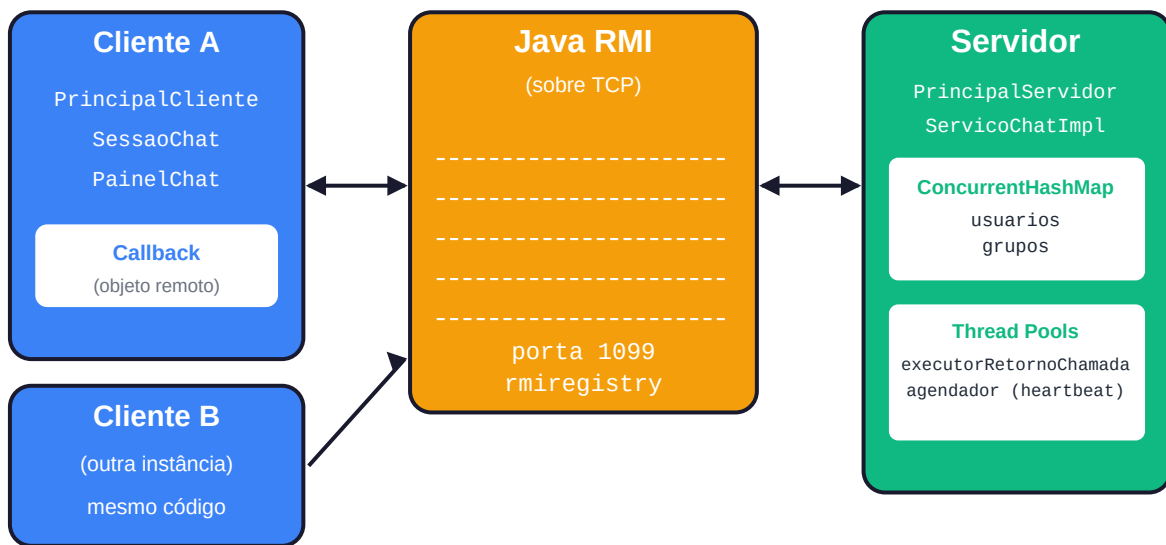


Figura 2 — Como cliente e servidor se conectam pela rede

O servidor abre a porta **1099** (padrão do RMI) e publica seu serviço com o nome **WhatsUTService**. Cada cliente faz lookup nesse nome, recebe um **stub** (objeto proxy) e a partir daí chama métodos como se fossem locais.

3. O que é RMI (explicação simplificada)

Imagine que você tem um método Java normal:

```
List<UsuarioDTO> usuarios = servico.obterUsuarios("luiz");
```

Esse servico parece um objeto comum. Mas com RMI, ele **não existe na sua máquina** — ele vive no processo do servidor. O método "viaja" pela rede, executa lá, e o resultado volta. Para você, programador, parece uma chamada local.

Por baixo dos panos

Quando você chama `servico.obterUsuarios(...)`, o RMI internamente faz quatro coisas:

- **Serializa** os argumentos — transforma os objetos em bytes (eles devem implementar `Serializable`).
- Envia esses bytes pelo **socket TCP** até o servidor.
- No servidor, **desserializa** os bytes e executa o método de verdade.
- Serializa o retorno e manda de volta pelo mesmo caminho.

Você não escreve `Socket`, `OutputStream` ou protocolo binário em lugar nenhum. **O RMI esconde a rede atrás de uma interface Java.**

Regras obrigatórias do RMI

- A interface remota **estende** `java.rmi.Remote`.
- Todo método declara `throws RemoteException` (a rede pode falhar).
- A implementação estende `UnicastRemoteObject`.
- Os DTOs trafegados implementam `Serializable`.

Veja no código do projeto (`common/ServicoChat.java`):

```
public interface ServicoChat extends Remote {
    UsuarioDTO entrar(String nomeUsuario, String senha,
        RetornoChamadaCliente retornoChamada)
        throws RemoteException;
    void enviarMensagemGrupo(String deUsuario, String idGrupo,
        MensagemDTO mensagem)
        throws RemoteException;
    // ... outros métodos ...
}
```

4. As classes principais

Vamos olhar quem é responsável por quê:

Classe	Pacote	O que faz
PrincipalServidor	server	main() do servidor. Abre o registry RMI e instancia o serviço.
ServicoChatImpl	server	Implementa todos os métodos RMI. É o cérebro do servidor.
UsuarioServidor	server	Modelo interno de usuário. Guarda hash BCrypt e o stub do callback.
GrupoServidor	server	Modelo interno de grupo. Mantém membros, histórico e política de admin.
InterfaceAdminServidor	server	Janela Swing de administração (usuários, grupos, configs, logs).
ServicoChat	common	Interface RMI principal. Contrato do servidor.
RetornoChamadaCliente	common	Interface RMI do cliente. Servidor chama estes métodos para 'empurrar' eventos.
UsuarioDTO, GrupoDTO	common	Objetos serializáveis que trafegam pela rede.
MensagemDTO	common	Representa uma mensagem (texto, arquivo ou sistema).
PrincipalCliente	client	main() do cliente. Cria a janela e gerencia login↔chat.
SessaoChat	client	Mantém conexão RMI, usuário logado e thread de heartbeat.
RetornoChamadaClienteImpl	client	Implementação do callback. Recebe eventos do servidor.
PainelChat, PainelLogin	client.ui	Telas Swing (login, chat, criação de grupos).

Por que dois modelos para usuário?

UsuarioDTO trafega pela rede e é seguro mostrar para qualquer cliente: contém só nome, status e flag de admin. Já UsuarioServidor vive apenas no servidor e contém **informações sensíveis** que não devem sair daqui: o hash BCrypt da senha e a referência remota ao callback do cliente. O método paraDTO() converte um no outro filtrando esses campos.

5. Fluxo de uma mensagem em grupo

Vamos rastrear o que acontece quando **alice** envia "oi" em um grupo onde **bob** também é membro:

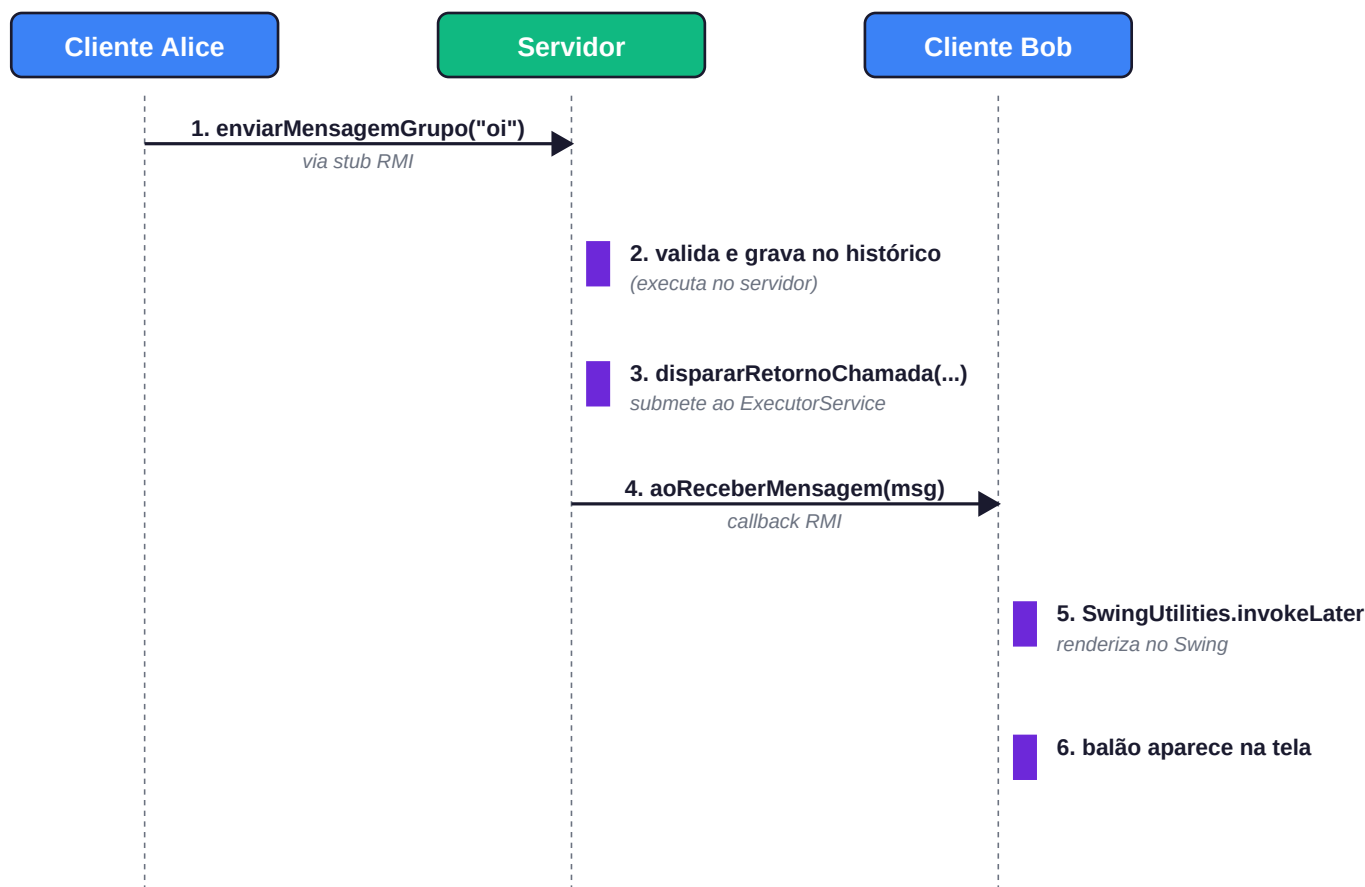


Figura 3 — Sequência de uma mensagem de grupo

Passo a passo no código

1. Em `PainelChat.enviarMensagem()`, o usuário aperta Enviar:

```

MensagemDTO msg = MensagemDTO.texto(meuParamario, meuExibicao,
idChatAtivo, chatAtivoEhGrupo, texto);
// ...
sessao.getServico().enviarMensagemGrupo(meuParamario, idChatAtivo, msg);

```

2. O RMI serializa msg e manda pelo socket. No servidor, `ServicoChatImpl.enviarMensagemGrupo` executa:

```

grupo.adicionarMensagem(mensagem); // grava no histórico em memória
for (String membro : grupo.getNomesMembros()) {
    if (membro.equals(deUsuario)) continue; // pula o remetente
    UsuarioServidor u = usuarios.get(membro);
    if (u != null && u.estaOnline()) {
        dispararRetornoChamada(() -> {
            try { u.getRetornoChamada().aoReceberMensagem(mensagem); }
            catch (RemoteException e) { tratarClienteMorto(...); }
        });
    }
}

```

```
}
```

3. O `dispararRetornoChamada` joga o trabalho num *thread pool* — assim o método retorna rapidamente para alice, e os callbacks rodam em paralelo. No cliente de bob, `RetornoChamadaClienteImpl.aoReceberMensagem` chama o listener que foi registrado em `PainelChat`:

```
cb.setAoReceberMensagem(msg -> SwingUtilities.invokeLater(() -> {  
    String chave = msg.isEhGrupo() ? msg.getIdDestino() : msg.getNomeRemetente();  
    if (chave.equals(idChatAtivo)) {  
        adicionarMensagem(msg); // desenha o balão  
    } else {  
        naoLidas.merge(chave, 1, Integer::sum); // badge  
    }  
}));
```


6. O padrão Callback (RMI bidirecional)

Esse é o conceito mais importante do projeto. Sem ele, o cliente teria que ficar perguntando "tem mensagem nova?" a cada segundo — uma técnica chamada **polling**, que é lenta e desperdiça rede.

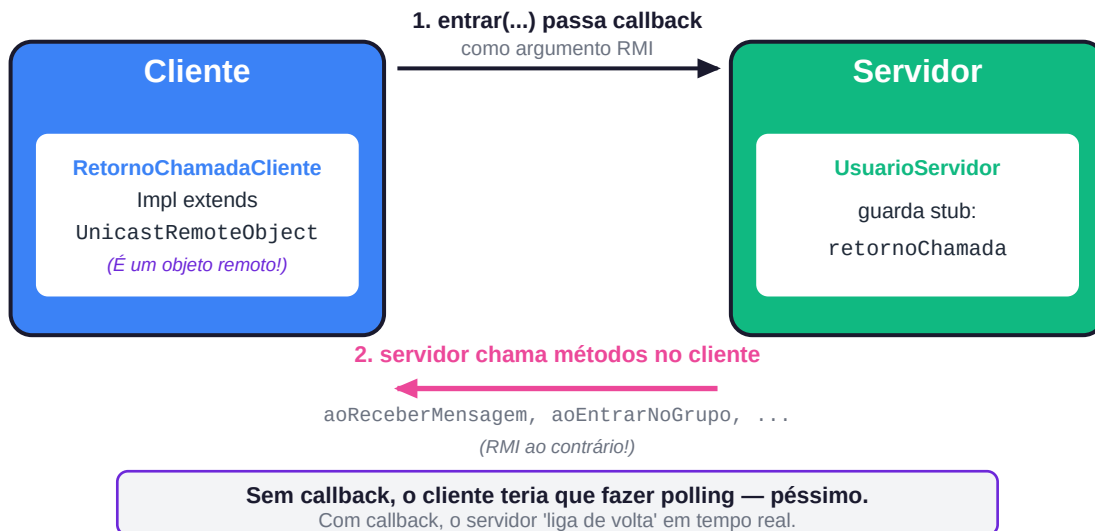


Figura 4 — Cliente é também um objeto remoto

Como funciona

1. No login, o cliente envia uma referência ao próprio callback:

```
// SessaoChat.entrar()
retornoChamada = new RetornoChamadaClienteImpl();
usuarioAtual = servico.entrar(nomeUsuario, senha, retornoChamada);
```

2. O servidor guarda essa referência no UsuarioServidor:

```
// ServicoChatImpl.entrar()
usuario.setRetornoChamada(retornoChamada);
```

3. A partir daí, sempre que algo acontece (mensagem nova, alguém entrou no grupo, ban do admin...), o servidor **chama métodos no cliente** usando essa referência. RMI *ao contrário*.

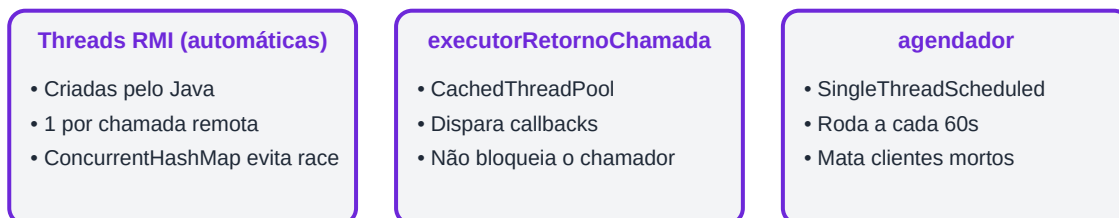
Os métodos disponíveis no callback estão em RetornoChamadaCliente:

- aoReceberMensagem — chegou mensagem nova
- aoAtualizarListaUsuarios — lista de usuários mudou
- aoMudarStatusUsuario — alguém ficou online/offline
- aoEntrarNoGrupo — você foi adicionado a um grupo
- aoReceberSolicitacao — alguém pediu para entrar (admin)
- aoAprovarSolicitacao — sua entrada foi aprovada
- aoSerBanidoDoServidor — você foi banido
- verificar — heartbeat: "você está vivo?"

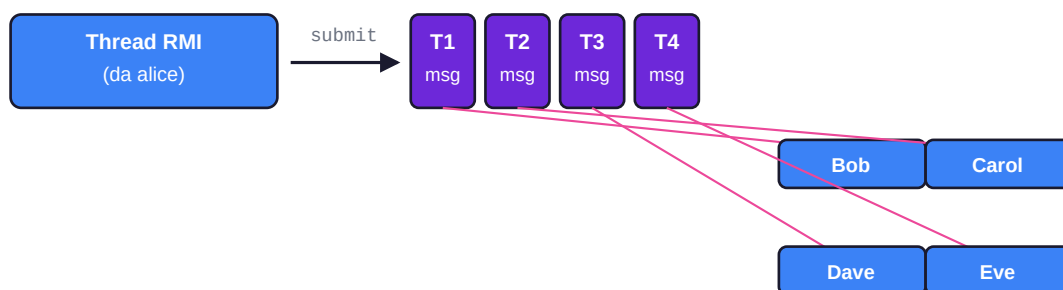
7. Threads — quem faz o quê

Sistemas distribuídos vivem de paralelismo. Veja onde threads são usadas no projeto:

Threads do servidor — quem faz o quê



Exemplo: alice envia mensagem ao grupo de 5 pessoas



Resultado: alice's sendGroupMessage retorna em milissegundos
mesmo se um dos destinos estiver lento — os callbacks rolam em paralelo.

Figura 5 — Como as threads colaboram numa mensagem de grupo

a) Threads automáticas do RMI

Isso é **de graça**. Quando 10 clientes chamam `obterUsuarios()` ao mesmo tempo, o RMI cria 10 threads no servidor automaticamente. Por isso o projeto usa `ConcurrentHashMap` em vez de `HashMap`:

```
private final ConcurrentHashMap<String, UsuarioServidor> usuarios
= new ConcurrentHashMap<>();
```

b) Pool de threads para callbacks

Quando o servidor entrega uma mensagem a 10 membros do grupo, ele não pode esperar 10 chamadas de rede sequencialmente. Em vez disso, joga cada callback num `CachedThreadPool`:

```
private final ExecutorService executorRetornoChamada =
Executors.newCachedThreadPool(r -> {
Thread t = new Thread(r, "trabalhador-retorno-chamada");
t.setDaemon(true);
return t;
});

private void dispararRetornoChamada(Runnable r) {
executorRetornoChamada.submit(r);
}
```

O `submit()` coloca a tarefa numa fila e retorna na hora. As threads do pool pegam e executam. Resultado: o método `enviarMensagemGrupo` retorna em milissegundos para alice, mesmo se um dos destinos estiver com a rede lenta.

c) Thread agendada do heartbeat

Um `ScheduledExecutorService` dispara uma verificação a cada 60 segundos para identificar clientes que não respondem mais:

```
agendador.scheduleAtFixedRate(this::verificarHeartbeats, 60, 60,
    TimeUnit.SECONDS);
```

d) Thread Timer no cliente

Cada cliente avisa o servidor "estou vivo" a cada 20 segundos, em `SessaoChat.iniciarHeartbeat()`:

```
timerHeartbeat.scheduleAtFixedRate(new TimerTask() {
    @Override public void run() {
        servico.manterConectado(usuarioAtual.getNomeUsuario());
    }
}, 20_000, 20_000);
```

e) `SwingUtilities.invokeLater`

Quando um callback chega no cliente, ele roda numa thread do RMI — **não** na thread de UI do Swing. Tocar em componentes Swing fora da EDT (Event Dispatch Thread) causa bugs sutis. Por isso todos os handlers usam:

```
cb.setAoReceberMensagem(msg -> SwingUtilities.invokeLater(() -> {
    // aqui é seguro mexer na UI
    adicionarMensagem(msg);
}));
```

8. Persistência e estado

Não há banco de dados. **Tudo é mantido em memória** no servidor, em três estruturas principais (todas ConcurrentHashMap):

- usuarios: nome → UsuarioServidor
- grupos: id → GrupoServidor
- historicosPrivados: chave "a:b" → fila de mensagens

A **consequência prática**: ao reiniciar o servidor, todos os usuários, grupos e mensagens são perdidos. Para um projeto acadêmico isso é OK, mas é uma limitação óbvia para produção. Adicionar persistência seria um próximo passo natural — salvando esses mapas em SQLite ou em arquivos JSON.

9. Heartbeat e detecção de clientes mortos

Um problema comum em sistemas RMI: o cliente fecha (ou perde a conexão) sem fazer logout. O servidor ainda tem a referência ao callback antigo, e tentar chamá-lo gera RemoteException.

A solução é em duas pontas:

Cliente (a cada 20s): envia ping com `servico.manterConectado(nomeUsuario)`, que atualiza o `ultimoHeartbeat` no servidor.

Servidor (a cada 60s): percorre todos os usuários online. Se algum não atualizou o heartbeat há mais de N minutos (configurável), o servidor tenta um `callback.verificar()`. Se a chamada falhar, `tratarClienteMorto()` remove o callback e marca offline.

```
// ServicoChatImpl.verificarHeartbeats()
for (UsuarioServidor u : usuarios.values()) {
    if (u.estaOnline() && u.getUltimoHeartbeat().isBefore(limite)) {
        try { u.getRetornoChamada().verificar(); }
        catch (RemoteException e) { tratarClienteMorto(u.getNomeUsuario()); }
    }
}
```

10. Como executar o sistema

Pré-requisitos

- Java 17 ou superior
- Maven 3.6 ou superior

Compilação

```
mvn clean package
```

Isso gera `target/whatsut-server.jar` — um JAR "fat" que contém todas as dependências (BCrypt, Gson) embutidas.

Servidor

```
java -cp target/whatsut-server.jar whatsut.server.PrincipalServidor
```

Inicia o `rmiregistry` embutido na porta 1099, registra o serviço como `WhatsUTService` e abre o painel admin. Login padrão: `admin / admin123`.

Cliente (uma janela por usuário)

```
java -cp target/whatsut-server.jar whatsut.client.PrincipalCliente
```

Você pode rodar várias instâncias para simular múltiplos usuários. Cada janela tem sua tela de login/cadastro, e depois a tela principal de chat com a sidebar (Usuários | Grupos).

Para criar um grupo:

- Usuário A: aba **Grupos** → botão **+ Novo Grupo** → preenche e cria
- Usuário B: aba **Grupos** → seção *Outros grupos disponíveis* → clica no grupo
- Usuário A: aceita o popup de solicitação
- Ambos veem mensagens em tempo real (callbacks RMI)

Em resumo: o WhatsUT mostra na prática como RMI esconde a complexidade de rede atrás de chamadas de método. As três ideias-chave para entender o projeto são (1) **interface remota** compartilhada entre cliente e servidor, (2) **registry** como agenda telefônica de serviços e (3) **callback RMI** para que o servidor possa empurrar eventos em tempo real, sem polling.